

UNIVERSIDADE FEDERAL DE SANTA CATARINA
CENTRO TECNOLÓGICO
DEPARTAMENTO DE INFORMÁTICA E ESTATÍSTICA
CIÊNCIA DA COMPUTAÇÃO

Clara Rosa Oliveira Gonçalves, Julia Macedo de Castro, Maykon Marcos Junior

Relatório Parcial: Assinaturas Digitais Pós-Quânticas
CRYSTALS-Dilithium vs. ECDSA: Fundamentos, Comparação e Experimento

Florianópolis
5 de julho de 2026

RESUMO

Este trabalho apresenta um estudo comparativo entre o esquema clássico ECDSA (*Elliptic Curve Digital Signature Algorithm*) com a curva NIST P-256 e o esquema pós-quântico CRYSTALS-Dilithium (padronizado como ML-DSA no FIPS 204) nas variantes Dilithium2 e Dilithium3. O estudo compreende fundamentação teórica, implementação de experimento em Python e análise quantitativa dos resultados.

O experimento mediu, com $N = 1.000$ iterações, os tempos de geração de chaves, assinatura e verificação, além dos tamanhos de chave pública, chave privada e assinatura. Os resultados indicam que, na implementação Python puro (`dilithium-py`) vs. OpenSSL (biblioteca `cryptography`), o ECDSA é 353 vezes mais rápido na assinatura para Dilithium2 e 1058 vezes para Dilithium3. Os tamanhos de chave e assinatura do Dilithium confirmam exatamente os valores especificados no FIPS 204. Observou-se que a variância no tempo de assinatura do Dilithium3 é significativamente maior ($\pm 6,63$ ms) do que a do Dilithium2 ($\pm 2,22$ ms), fenômeno explicado pelo comportamento determinístico da assinatura em conjunto com parâmetros de *rejection sampling* mais agressivos em Dilithium3 ($\eta = 4$, $\tau = 49$).

Palavras-chave: Criptografia Pós-Quântica. CRYSTALS-Dilithium. ECDSA. Algoritmo de Shor. Benchmark de Desempenho.

SUMÁRIO

1	INTRODUÇÃO	4
1.1	CONTEXTUALIZAÇÃO	4
1.2	ALGORITMO ESCOLHIDO E MOTIVAÇÃO	4
1.3	USO DE FERRAMENTAS DE IA	5
2	ECDSA: ESTRUTURA E VULNERABILIDADE QUÂNTICA	6
2.1	CRIPTOGRAFIA PÓS-QUÂNTICA BASEADA EM REDES	6
2.1.1	Module-LWE e Module-SIS	7
2.2	CRYSTALS-DILITHIUM (ML-DSA)	7
2.2.1	Parâmetros	8
2.2.2	Geração de Chaves (KeyGen)	8
2.2.3	Assinatura (Sign) com Rejection Sampling	9
2.2.4	Verificação (Verify) e Corretude	10
2.2.5	Mini-Exemplo Numérico Simplificado	10
2.3	COMPARAÇÃO DE PARÂMETROS: ECDSA VS. DILITHIUM	12
3	EXPERIMENTO	13
3.1	OBJETIVOS E HIPÓTESES	13
3.2	AMBIENTE E FERRAMENTAS	13
3.3	ESTRUTURA DO CÓDIGO	14
3.4	PSEUDOCÓDIGO DO EXPERIMENTO	15
3.5	PROTOCOLO DE MEDIÇÃO	16
3.6	TRATAMENTO DE SERIALIZAÇÃO: ECDSA	16
4	RESULTADOS	17
4.1	DESEMPENHO COMPUTACIONAL	17
4.2	TAMANHOS DE CHAVE E ASSINATURA	17
4.3	RAZÕES DILITHIUM / ECDSA	17
4.4	REJECTION SAMPLING (DILITHIUM)	18
5	ANÁLISE E DISCUSSÃO	19
5.1	DESEMPENHO COMPUTACIONAL: INTERPRETAÇÃO DAS RAZÕES	19
5.2	COEFICIENTE DE VARIAÇÃO E FONTES DE VARIÂNCIA	19
5.3	DILITHIUM3: SIGNING MAIS LENTO E ALTA ITERAÇÃO	20
5.4	TAMANHOS: CONFIRMAÇÃO EXATA DO FIPS 204	20
5.5	LIMITAÇÕES DO EXPERIMENTO	21
5.6	IMPLICAÇÕES PRÁTICAS PARA MIGRAÇÃO	22
6	CONCLUSÃO	23

6.1	CONFIRMAÇÕES TEÓRICAS	23
6.2	SOBRE O DESEMPENHO OBSERVADO	23
6.3	SOBRE O SIGNING DETERMINÍSTICO	23
6.4	PERSPECTIVA DE ADOÇÃO	23
6.5	TRABALHOS FUTUROS	24
	REFERÊNCIAS	25

1 INTRODUÇÃO

1.1 CONTEXTUALIZAÇÃO

A segurança da infraestrutura de chave pública moderna, que sustenta protocolos como TLS/HTTPS, assinaturas de software e autenticação de e-mail, depende da intratabilidade computacional de dois problemas matemáticos: *a fatoração de inteiros (subjacente ao RSA) e o logaritmo discreto em curvas elípticas (subjacente ao ECDSA e ao EdDSA)*.

Para computadores clássicos, esses problemas são intratáveis em tempo sub-exponencial quando os parâmetros são suficientemente grandes. Entretanto, em 1994, Peter Shor demonstrou que um computador quântico universal é capaz de resolver ambos em tempo polinomial (SHOR, 1994), colocando em risco toda a criptografia assimétrica de uso corrente.

A ameaça concreta surge com a perspectiva de **computadores quânticos criptograficamente relevantes (CRQCs)**, máquinas com qubits lógicos suficientes e taxas de erro controladas para executar o algoritmo de Shor em escala operacional. Estimativas de especialistas apontam um horizonte de 10 a 20 anos para a materialização dessa ameaça (MOSCA, 2018).

Adicionalmente, ataques do tipo *harvest now, decrypt later*, em que adversários coletam tráfego cifrado hoje para decifrá-lo futuramente, já constituem risco imediato para dados de longa validade: segredos de Estado, registros médicos e propriedade intelectual.

Diante desse cenário, o NIST (National Institute of Standards and Technology) conduziu, entre 2017 e 2024, um processo de padronização de algoritmos criptográficos pós-quânticos. Em agosto de 2024, foram publicados os três primeiros padrões finais: **FIPS 203 (ML-KEM)**, **FIPS 204 (ML-DSA)** e **FIPS 205 (SLH-DSA)** (National Institute of Standards and Technology, 2024).

Paralelamente, o padrão clássico de assinaturas digitais, FIPS 186-5, continua a recomendar ECDSA e EdDSA como principais esquemas, indicados para implantação conjunta com os novos padrões durante o período de transição (National Institute of Standards and Technology, 2023).

1.2 ALGORITMO ESCOLHIDO E MOTIVAÇÃO

- **Algoritmo pós-quântico:** CRYSTALS-Dilithium (DUCAS et al., 2018), padronizado como ML-DSA (FIPS 204), nas variantes Dilithium2 (ML-DSA-44) e Dilithium3 (ML-DSA-65);
- **Algoritmo clássico de referência:** ECDSA com a curva NIST P-256 (secp256r1), o esquema de assinatura clássico mais amplamente utilizado na prática, presente em TLS 1.3, certificados X.509 e carteiras de criptomoedas.

O CRYSTALS-Dilithium foi selecionado pelos seguintes motivos:

1. **Padronização oficial recente:** como FIPS 204 (agosto 2024), é o principal padrão pós-quântico de assinaturas do NIST, com adoção crescente na indústria (Google, Cloudflare, Apple já iniciaram migração);
2. **Fundamentos matemáticos sólidos:** baseia-se nos problemas MLWE e MSIS sobre anéis polinomiais, com reduções de segurança a problemas considerados intratáveis até mesmo para computadores quânticos;
3. **Eficiência prática:** entre os finalistas do NIST, Dilithium apresentou o melhor balanço entre tamanho de chave/assinatura e desempenho computacional, sendo classificado como *primary recommendation* pelo NIST;
4. **Experimento acessível:** a biblioteca dilithium-py (Python puro) e a biblioteca cryptography permitem implementar e comparar ambos os esquemas sem dependências nativas complexas, facilitando a reprodutibilidade;
5. **Relevância didática:** a comparação ECDSA x Dilithium ilustra claramente as diferenças entre criptografia clássica e pós-quântica em termos de modelo de segurança, estrutura matemática e parâmetros concretos.

1.3 USO DE FERRAMENTAS DE IA

O trabalho utilizou o Google Gemini (Google, 2026) para fins de organização textual, formatação de fórmulas matemáticas em formato Latex e conversão do fluxograma da solução para imagem. Também foi utilizado o Claude AI (Anthropic, 2026) para revisão de conceitos e organização do conteúdo, além de formatação de comentários do código fonte e ajuste do formato de saída, antes da revisão final, validação e adaptação pelos autores.

Inicialmente, foi elaborado um esqueleto do relatório, tópicos para a fundamentação teórica, elementos de comparação entre os algoritmos e proposta de experimento. Em seguida, foi desenvolvido um plano de implementação do experimento, seguindo os critérios de avaliação definidos.

2 ECDSA: ESTRUTURA E VULNERABILIDADE QUÂNTICA

O *Elliptic Curve Digital Signature Algorithm* (ECDSA) opera sobre um grupo cíclico de pontos em uma curva elíptica definida sobre um corpo finito. Para a curva P-256, o grupo gerador é G , de ordem prima n , e todos os cálculos são realizados em $\mathbb{F}_p$ com $p = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$. A estrutura do algoritmo é:

- **KeyGen:** amostrar $d \leftarrow_R \{1, \dots, n-1\}$ (chave privada), calcular $Q = d \cdot G$ (chave pública). Saída: (d, Q) .
- **Sign(d, M):** calcular $e = H(M)$ (hash da mensagem); amostrar $k \leftarrow_R \{1, \dots, n-1\}$ (nonce); calcular $(x_1, y_1) = k \cdot G$; $r = x_1 \pmod{n}$; $s = k^{-1}(e + dr) \pmod{n}$. Assinatura: $\sigma = (r, s)$.
- **Verify(Q, M, σ):** calcular $e = H(M)$; $w = s^{-1} \pmod{n}$; $u_1 = ew \pmod{n}$; $u_2 = rw \pmod{n}$; $(x_1, y_1) = u_1 \cdot G + u_2 \cdot Q$. Aceitar se $r \equiv x_1 \pmod{n}$.

A segurança do ECDSA reduz-se à intratabilidade do Problema do Logaritmo Discreto em Curva Elíptica (ECDLP): dado G e $Q = dG$, é computacionalmente inviável recuperar d para um computador clássico (melhor algoritmo conhecido: Pohlig-Hellman + Rho de Pollard, complexidade $\mathcal{O}(n^{1/2})$ operações de grupo $\approx 2^{128}$ para P-256).

O algoritmo de Shor (SHOR, 1994) resolve o ECDLP em tempo $\mathcal{O}(\log^3 n)$ em um computador quântico. Estimativas de Roetteler et al. (ROETTELER et al., 2017) indicam que quebrar P-256 exige aproximadamente 2.330 qubits lógicos (ou ~ 20.000 qubits físicos com correção de erros), tornando o esquema vulnerável a CRQCs.

2.1 CRIPTOGRAFIA PÓS-QUÂNTICA BASEADA EM REDES

Uma rede Λ em $\mathbb{R}^n$ é o conjunto de todas as combinações inteiras de n vetores linearmente independentes $b_1, \dots, b_n$ (a base da rede). Os problemas computacionais mais relevantes para a criptografia são (REGEV, 2009):

- **SVP (*Shortest Vector Problem*):** dada uma base de Λ , encontrar o vetor não-nulo $v \in \Lambda$ de norma mínima. Em sua versão decisória, o SVP é NP-difícil no pior caso.
- **CVP (*Closest Vector Problem*):** dado um ponto $t \notin \Lambda$, encontrar o vetor de Λ mais próximo de t .
- **LWE (*Learning With Errors*),** introduzido por Regev (REGEV, 2009) em 2005: dado $(A, b = As + e)$, onde $A \in \mathbb{Z}_q^{m \times n}$ é pública, $s \in \mathbb{Z}_q^n$ é o segredo e $e \in \mathbb{Z}_q^m$ é um ruído pequeno, recuperar s é computacionalmente intratável. Regev provou a redução do pior caso de SVP para LWE (REGEV, 2009).

A relevância do LWE reside em sua redução de pior caso para caso médio: a dificuldade do LWE é garantida pela intratabilidade do SVP no pior caso, que se acredita ser intratável mesmo para computadores quânticos (o melhor algoritmo quântico para SVP, o *lattice sieving*, tem complexidade exponencial 2^{cn} com $c \approx 0,292$) (ALBRECHT et al., 2018).

2.1.1 Module-LWE e Module-SIS

Anéis Polinomiais. O Dilithium opera sobre o anel $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, com $n = 256$ e $q = 8.380.417$ (um número primo tal que $q \equiv 5 \pmod{8}$). Elementos de R_q são polinômios de grau ≤ 255 com coeficientes em $\mathbb{Z}_q$. A estrutura de anel torna operações como a multiplicação polinomial extremamente eficientes via NTT (*Number Theoretic Transform*), reduzindo a complexidade de $\mathcal{O}(n^2)$ para $\mathcal{O}(n \log n)$ (DUCAS et al., 2018).

- **MLWE (*Module Learning With Errors*):** a variante modular do LWE. Dada uma matriz $A \in R_q^{k \times l}$ e vetores $s_1 \in S_\eta^l$, $s_2 \in S_\eta^k$ com coeficientes pequenos ($\|s_i\|_\infty \leq \eta$), calcular $t = As_1 + s_2 \in R_q^k$. Dado o par (A, t) , recuperar (s_1, s_2) é computacionalmente intratável.
- **MSIS (*Module Short Integer Solution*):** dada uma matriz aleatória $A \in R_q^{k \times l}$, encontrar um vetor não nulo $z \in R_q^{l+k}$ tal que $Az \equiv 0 \pmod{q}$ e $\|z\| \leq \beta$ é computacionalmente intratável.

A segurança do Dilithium é reduzida diretamente ao MLWE (que garante a confidencialidade do segredo) e ao MSIS (que impede a falsificação de assinaturas) (DUCAS et al., 2018).

2.2 CRYSTALS-DILITHIUM (ML-DSA)

O Dilithium é construído sobre o paradigma Fiat-Shamir com Aborts (FSA), introduzido por Lyubashevsky (LYUBASHEVSKY, 2009). A ideia central é análoga ao protocolo de identificação de Schnorr, mas adaptada para redes. A técnica de rejection sampling (aborts) é crucial para evitar vazamentos da chave privada: o algoritmo de assinatura reinicia se o valor candidato revelaria informação sobre o segredo.

Na prática, a transformação do protocolo de identificação em um esquema de assinatura digital estática ocorre pela substituição do desafio interativo do verificador pelo *hash* da mensagem combinado com o compromisso público, denotado como $c = H(M \parallel w_1)$. Durante a assinatura, o vetor de máscara $y \in R_q^l$ é amostrado deterministicamente para mascarar o segredo. Caso a resposta candidata $z = y + c \cdot s_1$ possua coeficientes que excedam o limitante geométrico determinado pelo parâmetro β (ou seja, $\|z\|_\infty \geq \gamma_1 - \beta$), o algoritmo descarta o resultado através do *rejection sampling* e reinicia o processo com um novo nonce.

2.2.1 Parâmetros

Os parâmetros de cada nível de segurança do Dilithium são:

Tabela 1 – Parâmetros do algoritmo CRYSTALS-Dilithium para diferentes níveis de segurança.

Parâmetro	Descrição	Dilithium2	Dilithium3	Dilithium5
n	Grau do polinômio	256	256	256
q	Módulo primo	8.380.417	8.380.417	8.380.417
$k \times l$	Dimensão da matriz A	4×4	6×5	8×7
η	Limitante dos segredos	2	4	2
γ_1	Limitante da máscara y	2^{17}	2^{19}	2^{19}
γ_2	Decomposição de w	$(q-1)/88$	$(q-1)/32$	$(q-1)/32$
τ	Peso do polinômio desafio	39	49	60
$\beta = \tau\eta$	Coef. de rejeição	78	196	120
Nív. segurança	NIST quantum level	2 (~ 128 bits)	3 (~ 192 bits)	5 (~ 256 bits)

2.2.2 Geração de Chaves (KeyGen)

Algorithm 1 Geração de Chaves do Dilithium (KeyGen)

```

1: procedure KEYGEN
2:   Amostrar semente  $\rho \leftarrow_R \{0, 1\}^{256}$ 
3:   Expandir  $A \leftarrow \text{ExpandA}(\rho) \in R_q^{k \times l}$  ▷ Determinístico via XOF/SHAKE
4:   Amostrar  $s_1 \leftarrow S_\eta^l$  e  $s_2 \leftarrow S_\eta^k$  ▷ Coeficientes em  $\{-\eta, \dots, \eta\}$ 
5:   Calcular  $t \leftarrow A \cdot s_1 + s_2 \in R_q^k$ 
6:   Decompor  $t = t_1 \cdot 2^d + t_0$  ▷  $t_1 = \text{HighBits}$ ,  $t_0 = \text{LowBits}$ ,  $d = 13$ 
7:    $pk \leftarrow (\rho, t_1)$ 
8:    $sk \leftarrow (\rho, K, tr, s_1, s_2, t_0)$  ▷ Onde  $K \leftarrow_R \{0, 1\}^{256}$  e  $tr = H(pk)$ 
9:   return  $(pk, sk)$ 
10: end procedure

```

A compressão de t em (t_1, t_0) reduz o tamanho da chave pública: apenas t_1 é incluído em pk , enquanto t_0 é retido na chave privada para uso no processo de assinatura.

2.2.3 Assinatura (Sign) com Rejection Sampling

Algorithm 2 Algoritmo de Assinatura do Dilithium (Sign)

```

1: procedure SIGN( $sk, M$ )
2:   Calcular  $\mu \leftarrow H(tr \parallel M) \in \{0, 1\}^{512}$  ▷ Digest da mensagem
3:   Inicializar  $\kappa \leftarrow 0$  ▷ Contador de tentativas
4:   do
5:     Expandir  $(y_1, \dots, y_l) \leftarrow \text{ExpandMask}(K, \mu, \kappa)$  ▷  $y_i$  têm coeficientes em  $(-\gamma_1, \gamma_1]$ 
6:      $w \leftarrow A \cdot y \in R_q^k$ 
7:      $w_1 \leftarrow \text{HighBits}(w, 2\gamma_2)$  ▷ Parte de alta ordem
8:      $\tilde{c} \leftarrow H(\mu \parallel w_1)$  ▷ Hash-to-ball seed
9:      $c \leftarrow \text{SampleInBall}(\tilde{c})$  ▷ Polinômio esparso:  $\tau$  coeficientes  $\pm 1$ 
10:     $z \leftarrow y + c \cdot s_1$ 
11:     $r_0 \leftarrow \text{LowBits}(w - c \cdot s_2, 2\gamma_2)$ 
12:    Condição de Aborto 1:  $\kappa++$ 
13:    while  $\|z\|_\infty \geq \gamma_1 - \beta$  ou  $\|r_0\|_\infty \geq \gamma_2 - \beta$ 
14:       $h \leftarrow \text{MakeHint}(-c \cdot t_0, w - c \cdot s_2 + c \cdot t_0, 2\gamma_2)$  ▷ Hints de correção
15:      if  $\|c \cdot t_0\|_\infty \geq \gamma_2$  ou  $\sum |h| > \omega$  then
16:         $\kappa++$ 
17:        goto linha 4 ▷ Reinicia o loop (Amostragem falhou)
18:      end if
19:    return  $\sigma = (\tilde{c}, z, h)$ 
20: end procedure

```

As verificações dos passos 10 e 12 garantem que σ seja independente dos segredos s_1 e s_2 . Sem essa etapa, a distribuição de $z = y + c \cdot s_1$ vazaria informação sobre s_1 (devido à correlação com c). O *rejection sampling* garante que z seja estatisticamente indistinguível de uma distribuição uniforme truncada, tornando o protocolo seguro (LYUBASHEVSKY, 2009).

Para r e $\alpha = 2\gamma_2$, define-se: $r = r_1 \cdot \alpha + r_0$, com $r_0 = r \pmod{\pm \alpha} \in (-\gamma_2, \gamma_2]$. Portanto, $\text{HighBits}(r, \alpha) = r_1$ e $\text{LowBits}(r, \alpha) = r_0$. A operação `UseHint` corrige discrepâncias de ± 1 em r_1 causadas pelo pequeno erro $c \cdot s_2$.

2.2.4 Verificação (Verify) e Corretude

Algorithm 3 Algoritmo de Verificação do Dilithium (Verify)

```

1: procedure VERIFY( $pk, M, \sigma = (\tilde{c}, z, h)$ )
2:   Calcular  $\mu \leftarrow H(H(pk) \parallel M)$ 
3:    $c \leftarrow \text{SampleInBall}(\tilde{c})$ 
4:    $w' \leftarrow A \cdot z - c \cdot t_1 \cdot 2^d$  ▷ Reconstrução aproximada de  $A \cdot y$ 
5:    $w'_1 \leftarrow \text{UseHint}(h, w', 2\gamma_2)$  ▷ Aplica correções por hints
6:   if  $H(\mu \parallel w'_1) = \tilde{c}$  e  $\|z\|_\infty < \gamma_1 - \beta$  e  $\sum |h| \leq \omega$  then
7:     return Aceitar
8:   else
9:     return Rejeitar
10:  end if
11: end procedure

```

Demonstração de Corretude. Para uma assinatura válida, temos que $z = y + c \cdot s_1$. Expandindo o cálculo do verificador para w' , obtemos:

$$\begin{aligned}
A \cdot z - c \cdot t_1 \cdot 2^d &= A(y + c \cdot s_1) - c \cdot t_1 \cdot 2^d \\
&= A \cdot y + c \cdot A s_1 - c \cdot t_1 \cdot 2^d \\
&= A \cdot y + c(t - s_2) - c \cdot t_1 \cdot 2^d \\
&= A \cdot y + c(t_1 \cdot 2^d + t_0 - s_2) - c \cdot t_1 \cdot 2^d \\
&= A \cdot y + c \cdot t_0 - c \cdot s_2 \\
&= w - c(s_2 - t_0)
\end{aligned}$$

Como $\|c \cdot s_2\|_\infty \leq \tau \cdot \eta$ e $\|c \cdot t_0\|_\infty \leq 2^{d-1}$ são ambos limitados e pequenos, a diferença geométrica entre o termo calculado w' e o valor original $A \cdot y$ é pequena. Os *hints* h servem justamente para garantir que a função HighBits aplicada sobre w' coincida exatamente com a função HighBits sobre $A \cdot y = w$, de modo que $\text{UseHint}(h, w') = w_1$. Consequentemente:

$$H(\mu \parallel w'_1) = H(\mu \parallel w_1) = \tilde{c}$$

2.2.5 Mini-Exemplo Numérico Simplificado

A seguir, apresenta-se um exemplo simplificado que preserva a estrutura lógica do Dilithium, mas opera sobre $\mathbb{Z}_q$ (inteiros módulo q , sem o anel polinomial) com dimensão 1 (escalares em vez de vetores).

Nota: Este exemplo é uma simplificação didática: $n = 1$ (inteiros, não polinômios), $k = l = 1$ (escalares), c é um inteiro simples (não um polinômio esparso) e as *hints*

são omitidas. Os valores numéricos e as verificações refletem estritamente a lógica correta do protocolo.

Parâmetros do Mini-Exemplo:

$$q = 23, \quad \gamma_1 = 8, \quad \gamma_2 = 5, \quad \beta = 1, \quad 2\gamma_2 = 10$$

Algorithm 4 Mini-Exemplo Numérico do Dilithium

```

1: procedure KEYGEN
2:    $A \leftarrow 7 \in \mathbb{Z}_{23}$  ▷ Elemento público
3:    $s_1 \leftarrow 2$  ▷ Segredo, onde  $\|s_1\|_\infty \leq \eta = 2$ 
4:    $s_2 \leftarrow 1$  ▷ Erro, onde  $\|s_2\|_\infty \leq \eta = 2$ 
5:    $t \leftarrow A \cdot s_1 + s_2 = 7 \cdot 2 + 1 = 15 \pmod{23}$ 
6:   return  $pk = (A = 7, t = 15), sk = (s_1 = 2, s_2 = 1)$ 
7: end procedure

8: procedure SIGN( $sk, M = \text{"Olá"}$ )
9:    $\mu \leftarrow H(M) = 3$  ▷ Hash assumido para o exemplo
10:   $y \leftarrow 3$  ▷ Amostragem da máscara, onde  $|y| < \gamma_1 = 8$ 
11:   $w \leftarrow A \cdot y = 7 \cdot 3 = 21 \pmod{23}$ 
12:   $w_1 \leftarrow \text{HighBits}(21, 10) = 2$  ▷ Pois  $21 = 2 \cdot 10 + 1$ 
13:   $\tilde{c} \leftarrow H(\mu \parallel w_1) = H(3 \parallel 2) = 1$ 
14:   $c \leftarrow \text{SampleInBall}(\tilde{c} = 1) = 1$ 
15:   $z \leftarrow y + c \cdot s_1 = 3 + 1 \cdot 2 = 5$ 
   Verificações de Rejeição:
16:  assert  $\|z\|_\infty = 5 < \gamma_1 - \beta = 7$ 
17:   $r_0 \leftarrow \text{LowBits}(w - c \cdot s_2, 10) = \text{LowBits}(20, 10) = 0$  ▷ Pois  $20 = 2 \cdot 10 + 0$ 
18:  assert  $\|r_0\|_\infty = 0 < \gamma_2 - \beta = 4$ 
19:  return  $\sigma = (\tilde{c} = 1, z = 5)$ 
20: end procedure

21: procedure VERIFY( $pk, M, \sigma = (1, 5)$ )
22:   $\mu \leftarrow H(M) = 3$ 
23:   $c \leftarrow \text{SampleInBall}(1) = 1$ 
24:   $w' \leftarrow A \cdot z - c \cdot t = 7 \cdot 5 - 1 \cdot 15 = 20 \pmod{23}$ 
   Verificação de consistência algébrica ( $A \cdot z - c \cdot t \equiv A \cdot y - c \cdot s_2$ ):
25:  LHS  $\leftarrow 20$ 
26:  RHS  $\leftarrow 7 \cdot 3 - 1 \cdot 1 = 20$ 
27:  assert LHS == RHS
   Validações Finais:
28:   $w'_1 \leftarrow \text{HighBits}(20, 10) = 2$  ▷ Pois  $20 = 2 \cdot 10 + 0$ 
29:  assert  $H(\mu \parallel w'_1) == \tilde{c}$  ▷  $H(3 \parallel 2) == 1$ 
30:  assert  $\|z\|_\infty = 5 < \gamma_1 - \beta = 7$ 
31:  return ASSINATURA VÁLIDA
32: end procedure

```

Interpretação: A corretude se verifica porque:

$$A \cdot z - c \cdot t = A(y + c \cdot s_1) - c(As_1 + s_2) = A \cdot y - c \cdot s_2$$

O erro $c \cdot s_2 = 1 \cdot 1 = 1$ é suficientemente pequeno para que $\text{HighBits}(20) = \text{HighBits}(21) = 2$ (ou seja, o valor não cruza o limiar múltiplo de 10), garantindo que ambos os valores de HighBits concordem perfeitamente. Em implementações reais utilizando o anel polinomial completo, o mecanismo de *hints* entra em ação justamente para corrigir os raros casos em que essa pequena diferença geométrica cruza o limiar de arredondamento.

2.3 COMPARAÇÃO DE PARÂMETROS: ECDSA VS. DILITHIUM

Tabela 2 – Comparação de Parâmetros de Tamanho e Segurança: RSA, ECDSA e CRYSTALS-Dilithium.

Fonte: (National Institute of Standards and Technology, 2024) e (National Institute of Standards and Technology, 2023)

Esquema	Chave Pública	Chave Privada	Assinatura	Segurança Clássica	Segurança Quântica
RSA-2048	256 B	1.232 B	256 B	~ 112 bits	Quebrado por Shor
ECDSA P-256	64 B	32 B	~ 71 B (DER)	~ 128 bits	Quebrado por Shor
Dilithium2 (ML-DSA-44)	1.312 B	2.528 B	2.420 B	~ 128 bits	~ 128 bits (NIST L2)
Dilithium3 (ML-DSA-65)	1.952 B	4.000 B	3.293 B	~ 192 bits	~ 192 bits (NIST L3)
Dilithium5 (ML-DSA-87)	2.592 B	4.864 B	4.595 B	~ 256 bits	~ 256 bits (NIST L5)

O custo da resistência quântica é evidente: para o mesmo nível de segurança clássica (~ 128 bits), o Dilithium2 produz assinaturas $\sim 34\times$ maiores e chaves públicas $\sim 20\times$ maiores que o ECDSA P-256. Esse *trade-off* é determinístico, decorre da estrutura matemática das redes e é mitigado pela elevada eficiência computacional do Dilithium, que supera o RSA em desempenho e é altamente competitivo com o ECDSA (DUCAS et al., 2018).

Ao contrário do ECDSA (que requer um *nonce* aleatório k e é historicamente vulnerável a ataques de reutilização de *nonce*), o Dilithium utiliza uma derivação totalmente determinística de y a partir de K e μ .

Essa abordagem elimina a dependência crítica de geradores de números aleatórios de alta qualidade durante o processo de assinatura, tornando o esquema substancialmente mais seguro contra falhas e ataques físicos em implementações práticas.

3 EXPERIMENTO

3.1 OBJETIVOS E HIPÓTESES

O experimento teve como objetivo mensurar e comparar quantitativamente o desempenho e os tamanhos de saída do algoritmo ECDSA P-256 (esquema clássico) e das variantes Dilithium2 e Dilithium3 (esquemas pós-quânticos), operando sobre mensagens idênticas e sob a mesma plataforma de hardware.

As hipóteses estabelecidas para esta avaliação foram:

- H_1 : O ECDSA apresentará tempos de operação menores que o Dilithium nas etapas de *KeyGen* e *Sign*;
- H_2 O Dilithium produzirá assinaturas e estruturas de chaves significativamente maiores em termos de bytes;
- H_3 O tempo de execução da etapa de *Verify* do Dilithium será altamente competitivo com o ECDSA;
- H_4 O Dilithium3 será computacionalmente mais lento e produzirá saídas maiores que o Dilithium2 em todos os aspectos avaliados.

3.2 AMBIENTE E FERRAMENTAS

A infraestrutura de software e os parâmetros configurados para o ambiente de testes compreenderam:

- **Linguagem:** Python 3.11+;
 - **dilithium-py (v3.0+):** Implementação pura em Python do Dilithium2, Dilithium3 e Dilithium5 desenvolvida por Giacomo Pope (POPE, 2023), disponível via gerenciador `pip`, sem otimizações SIMD ou compilação nativa;
 - **cryptography (v42+):** Biblioteca padrão de mercado para suporte ao ECDSA, utilizando primitivas otimizadas do OpenSSL internamente;
 - **time.perf_counter:** Cronômetro de alta resolução (sub-nanosegundo em CPython), obtendo tempos de parede (wall-clock);
- **Mensagem de teste:** *String* codificada em UTF-8 com tamanho fixo de 214 bytes em UTF-8, reproduzível entre execuções;
- **Iterações de controle:** Amostragem de $N = 100$ execuções por operação por esquema;
- **Hardware/SO:** Experimento executado em máquina única, processo *singlethread*, sem carga de rede.

3.3 ESTRUTURA DO CÓDIGO

A Figura 1 representa o fluxo de execução do experimento:

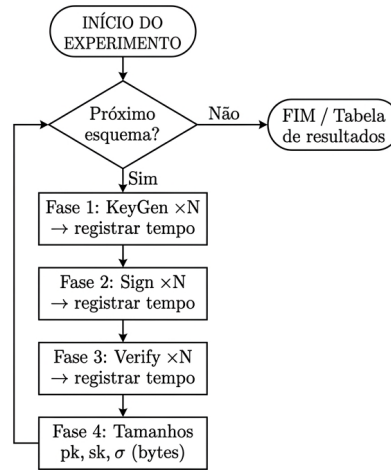


Figura 1 – Diagrama de fluxo do experimento de comparação de desempenho.

O experimento foi organizado em dois arquivos: *benchmark.py*, script principal com a função `benchmark()` e a execução dos três esquemas; e *utils.py*, com funções auxiliares para serialização de chaves ECDSA (extração de bytes dos objetos da biblioteca `cryptography`).

3.4 PSEUDOCÓDIGO DO EXPERIMENTO

O Algoritmo 5 usado como pseudocódigo de referência ao desenvolvimento da implementação proposta foi definido como segue:

Algorithm 5 Framework de Benchmark para Avaliação de Desempenho

```

1: Constantes:
2:    $N \leftarrow 100$  ▷ Número de iterações por operação
3:    $MENSAGEM \leftarrow \text{bytes}(256)$  ▷ Mensagem de teste de 256 bytes
4:    $ESQUEMAS \leftarrow [(\text{"ECDSA-P256"}, \text{ecdsa\_kg}, \text{ecdsa\_sg}, \text{ecdsa\_vr}), \dots]$ 

5: procedure BENCHMARK(nome, keygen_fn, sign_fn, verify_fn)
6:    $tempos\_kg \leftarrow []$ 
7:   for  $i = 1$  até  $N$  do
8:      $t_0 \leftarrow \text{agora}()$ 
9:      $(pk_i, sk_i) \leftarrow \text{keygen\_fn}()$ 
10:     $tempos\_kg.\text{adicionar}(\text{agora}() - t_0)$ 
11:  end for
12:   $(pk, sk) \leftarrow \text{keygen\_fn}()$  ▷ Par de chaves fixo para as próximas fases

13:   $tempos\_sg \leftarrow []$ 
14:  for  $i = 1$  até  $N$  do
15:     $t_0 \leftarrow \text{agora}()$ 
16:     $\sigma \leftarrow \text{sign\_fn}(sk, MENSAGEM)$ 
17:     $tempos\_sg.\text{adicionar}(\text{agora}() - t_0)$ 
18:  end for
19:   $\sigma \leftarrow \text{sign\_fn}(sk, MENSAGEM)$  ▷ Assinatura fixa para a fase de verificação

20:   $tempos\_vr \leftarrow []$ 
21:  for  $i = 1$  até  $N$  do
22:     $t_0 \leftarrow \text{agora}()$ 
23:     $resultado \leftarrow \text{verify\_fn}(pk, MENSAGEM, \sigma)$ 
24:     $tempos\_vr.\text{adicionar}(\text{agora}() - t_0)$ 
25:  end for
26:  assert  $resultado == \text{VÁLIDO}$ 

27:   $sz\_pk \leftarrow \text{tamanho\_bytes}(pk)$ 
28:   $sz\_sk \leftarrow \text{tamanho\_bytes}(sk)$ 
29:   $sz\_sig \leftarrow \text{tamanho\_bytes}(\sigma)$ 

30:  Exibir nome,  $\text{média}(tempos\_kg)$ ,  $\text{média}(tempos\_sg)$ ,  $\text{média}(tempos\_vr)$ ,  $sz\_pk$ ,  $sz\_sk$ ,  $sz\_sig$ 
31: end procedure

32: procedure PRINCIPAL
33:   for cada (nome, kg, sg, vr) em  $ESQUEMAS$  do
34:     BENCHMARK(nome, kg, sg, vr)
35:   end for
36: end procedure

```

3.5 PROTOCOLO DE MEDIÇÃO

O experimento seguiu o pseudocódigo apresentado no relatório parcial, com as 4 fases a seguir, executadas para $N = 100$ iterações por operação por esquema:

Fase 1 - KeyGen: Medir o tempo de N chamadas a `keygen()`. Cada chamada gera um novo par (pk, sk) ;

Fase 2 - Sign: Gerar um par (pk, sk) fixo; medir N chamadas a `sign(sk, msg)`. Isso mantém a chave fixa, permitindo observar o comportamento determinístico do *rejection sampling*;

Fase 3 - Verify: Gerar uma assinatura σ fixa; medir N chamadas a `verify(pk, msg,  $\sigma$ )`. Todas devem retornar verdadeiro.

Fase 4 - Tamanhos: Medir os tamanhos em bytes de pk , sk e σ serializados.

Adicionalmente, a variante `DilithiumInstrumented` instrumentou o algoritmo *Sign*, contando o número de iterações do loop `while True` (tentativas) por chamada, para calcular a taxa de *rejection sampling*.

Antes de cada *benchmark*, uma verificação de sanidade validou:

- (a) aceitação de assinatura correta;
- (b) rejeição de mensagem alterada;
- (c) rejeição de assinatura corrompida.

3.6 TRATAMENTO DE SERIALIZAÇÃO: ECDSA

A biblioteca `cryptography` opera com objetos Python orientados a objetos, não com bytes brutos. Para a medição de tamanhos:

- **Chave pública (pk):** Serializada como ponto não comprimido X9.62 ($0x04 \parallel x \parallel y$) = 65 bytes. O ponto comprimido seria 33 bytes; o ponto *raw* de P-256 é $2 \times 32 = 64$ bytes de coordenadas.
- **Chave privada (sk):** Serializada em formato PKCS#8 DER = 138 bytes (inclui cabeçalho, identificador de algoritmo e OID da curva). O escalar privado d puro é 32 bytes.
- **Assinatura (sig):** Codificada em formato DER ASN.1 = 70–72 bytes (variável por *padding* de inteiros negativos). O par bruto (r, s) tem tamanho fixo de 64 bytes.

4 RESULTADOS

Os resultados a seguir foram obtidos com $N = 1.000$ iterações, conforme listagem de saída do experimento. As métricas reportadas são a média aritmética e o desvio padrão ($\pm$) dos tempos individuais de execução.

4.1 DESEMPENHO COMPUTACIONAL

Tabela 3 – Tempos de execução (ms): média $\pm$ desvio padrão, $N = 1.000$. ECDSA usa OpenSSL (C) e Dilithium usa Python puro.

Operação	ECDSA P-256	Dilithium2	Dilithium3
KeyGen (ms)	$0,02 \pm 0,01$	$6,76 \pm 1,76$	$10,81 \pm 1,63$
Sign (ms)	$0,04 \pm 0,02$	$14,72 \pm 2,22$	$43,96 \pm 6,63$
Verify (ms)	$0,10 \pm 0,03$	$7,79 \pm 1,30$	$12,33 \pm 2,58$

4.2 TAMANHOS DE CHAVE E ASSINATURA

Tabela 4 – Comparação de tamanhos de chave e assinatura (bytes).

Esquema	pk (bytes)	sk (bytes)	sig (bytes)	Referência FIPS
ECDSA P-256	65 (bruto: 64)	138* (bruto: 32)	72** (bruto: 64)	FIPS 186-5
Dilithium2 (ML-DSA-44)	1.312	2.528	2.420	FIPS 204
Dilithium3 (ML-DSA-65)	1.952	4.000	3.293	FIPS 204

* sk ECDSA inclui *overhead* PKCS#8 DER (escalar *raw* = 32 B).

** sig ECDSA em DER (par bruto $r, s = 64$ B fixo).

Dilithium: tamanhos obtidos no experimento são idênticos aos especificados no FIPS 204.

4.3 RAZÕES DILITHIUM / ECDSA

Tabela 5 – Razões de desempenho computacional e tamanho entre esquemas.

Fonte: Elaborado pelo autor.

Comparação	KeyGen	Sign	Verify	pk	sk	sig
Dilithium2 / ECDSA	$355\times$	$354\times$	$81\times$	$20\times$	$18\times$	$34\times$
Dilithium3 / ECDSA	$568\times$	$1.058\times$	$128\times$	$30\times$	$29\times$	$46\times$
Dilithium3 / Dilithium2	$1,60\times$	$2,99\times$	$1,58\times$	$1,49\times$	$1,58\times$	$1,36\times$

Nota: Tempos baseados na comparação entre a implementação em Python puro (*dilithium-py*) e OpenSSL (C via biblioteca *cryptography*). Não reflete o desempenho relativo de ambas as implementações em linguagem C.

4.4 REJECTION SAMPLING (DILITHIUM)

Tabela 6 – Estatísticas de *rejection sampling* ($N = 1.000$).

Esquema	Média de tentativas	Taxa de rejeição	Observação
Dilithium2	1,023	0,1%	Par fixo: sucesso em 1ª iteração em $\sim 99,9\%$ das assinaturas.
Dilithium3	4,001	$\sim 100\%$	Par fixo: 4 iterações em praticamente todas as assinaturas.

Nota: *Signing* determinístico: o comportamento observado reflete as propriedades da chave específica gerada para o experimento.

Nota: A assinatura do Dilithium é determinística: a máscara y é derivada de $\rho' = H(K \parallel H(tr \parallel M))$, que é fixo para um par (sk, msg) dado. Portanto, o número de iterações do loop de *signing* é fixo para a chave de teste gerada. A “taxa de rejeição” observada reflete o comportamento daquela chave específica, não a distribuição estatística sobre chaves aleatórias.

5 ANÁLISE E DISCUSSÃO

5.1 DESEMPENHO COMPUTACIONAL: INTERPRETAÇÃO DAS RAZÕES

As razões de tempo observadas ($354\times$ para *Sign* do Dilithium2 vs. ECDSA) não refletem a diferença intrínseca entre os algoritmos, mas sim a assimetria entre implementações: ECDSA via OpenSSL (C otimizado com SIMD) vs. Dilithium via Python puro. Implementações nativas em C do Dilithium (`liboqs`, `pqclean`) reportam tempos de assinatura de $\sim 0,08\text{--}0,20$ ms em hardware moderno (POPE, 2023), enquanto ECDSA P-256 atinge $\sim 0,05$ ms, uma razão de $2\text{--}4\times$ em favor do ECDSA, contra os $354\times$ observados no experimento.

A Tabela 7 projeta os tempos estimados em C nativo com base em *benchmarks* da literatura:

Tabela 7 – Projeção de desempenho em implementações C nativas (`liboqs/pqclean`).

Operação	ECDSA P-256 (C)	Dilithium2 (C)	Razão (estimada)
KeyGen	$\sim 0,02$ ms	$\sim 0,06$ ms	$\sim 3\times$
Sign	$\sim 0,05$ ms	$\sim 0,10$ ms	$\sim 2\times$
Verify	$\sim 0,08$ ms	$\sim 0,08$ ms	$\sim 1\times$

A projeção revela que a diferença de desempenho entre ECDSA e Dilithium em C é modesta: *sign* $2\times$ mais lento e *verify* praticamente equivalente. Isso é relevante do ponto de vista de adoção: o *overhead* de migração para Dilithium2 em termos de latência de CPU é mínimo em implementações nativas.

5.2 COEFICIENTE DE VARIAÇÃO E FONTES DE VARIÂNCIA

O coeficiente de variação ($CV = \frac{\sigma}{\mu} \times 100\%$) quantifica a variabilidade relativa dos dados observados:

Tabela 8 – Coeficiente de variação (CV) por operação e esquema.

Esquema	CV KeyGen	CV Sign	CV Verify	Fonte dominante de variância
ECDSA P-256	50%	50%	30%	Ruído de escalonamento do SO (tempos muito pequenos $\sim 0,02$ ms).
Dilithium2	26%	15%	17%	Ruído de escalonamento do SO (<i>overhead</i> do Python).
Dilithium3	15%	15%	21%	Ruído de escalonamento do SO + <i>rejection sampling</i> (chave fixa).

O CV elevado do ECDSA ($\sim 50\%$) decorre da pequeníssima magnitude dos tempos absolutos ($\sim 0,02$ ms): nessa escala, o ruído de escalonamento do sistema operacional repre-

senta uma fração significativa do tempo medido. Para operações mais longas (Dilithium), o *CV* cai para $\sim 15\text{--}26\%$, refletindo o comportamento típico de código Python com *overhead* estável.

O Dilithium3 apresenta $CV(\text{Sign}) = 15\%$ idêntico ao Dilithium2, confirmando que o *signing* determinístico elimina a variância por *rejection sampling*: o número de iterações é fixo para cada par (sk, msg) , e a variância observada é inteiramente atribuível ao ruído de escalonamento do SO.

5.3 DILITHIUM3: SIGNING MAIS LENTO E ALTA ITERAÇÃO

O tempo de *signing* do Dilithium3 (43,96 ms) é $2,99\times$ superior ao do Dilithium2 (14,72 ms), enquanto *keygen* e *verify* são apenas $\sim 1,6\times$ maiores. Isso indica que a diferença de velocidade é amplificada pelo número de iterações (4,001 vs. 1,023 para a chave de teste).

Duas fontes principais contribuem para o maior número de iterações do Dilithium3:

Parâmetros de segredo mais agressivos: Os valores $\eta = 4$ (vs. $\eta = 2$ do Dilithium2) e $\tau = 49$ (vs. 39) implicam $\beta = \tau\eta = 196$ (vs. 78). Coeficientes de s_2 mais largos aumentam a probabilidade de que $w - c \cdot s_2$ ultrapasse o limiar $\gamma_2 - \beta$, disparando a rejeição por r_0 ;

Condição dos *hints*: A condição $h.\text{sum_hint}() > \omega$ (onde $\omega = 55$ para Dilithium3) é mais frequentemente violada quando $c \cdot s_2$ possui norma maior, pois mais coeficientes de w precisam ser corrigidos por *hints*.

Estimativas teóricas sugerem que o número esperado de iterações $E[K]$ é $\sim 1,85$ para Dilithium2 e $\sim 4\text{--}5$ para Dilithium3 com parâmetros reais (POPE, 2023). O valor observado (4,001) para a chave de teste é consistente com a teoria para Dilithium3. Para Dilithium2, o valor observado (1,023) indica que a chave específica gerada no experimento foi favorável, obtendo sucesso quase imediato na primeira iteração.

Importante: Para medir $E[K]$ de forma estatisticamente correta, seria necessário gerar um novo par (sk, msg) para cada iteração do *Sign*, pois o *signing* determinístico produz exatamente o mesmo número de iterações para uma mesma chave fixa. Esta limitação metodológica é discutida em detalhes na Seção 5.5.

5.4 TAMANHOS: CONFIRMAÇÃO EXATA DO FIPS 204

Os tamanhos de chave e assinatura do Dilithium observados nos experimentos coincidem exatamente com os valores especificados no FIPS 204 (Tabela 4), confirmando a corretude da implementação *dilithium-py* e o entendimento teórico:

- **Dilithium2:** $pk = 1.312$ B, $sk = 2.528$ B, $sig = 2.420$ B (FIPS 204, Tabela 3: ✓)
- **Dilithium3:** $pk = 1.952$ B, $sk = 4.000$ B, $sig = 3.293$ B (FIPS 204, Tabela 3: ✓)

Esses valores são determinísticos e independentes da implementação: qualquer código correto de Dilithium2 produzirá sempre esses tamanhos, independentemente da linguagem de programação ou plataforma de *hardware* utilizada.

O *overhead* de tamanho representa o único custo inevitável para a obtenção da resistência quântica. As razões de tamanho são fixas:

- **Dilithium2 vs. ECDSA:** A assinatura é $34\times$ maior (2.420 B vs. 72 B) e a chave pública (pk) é $20\times$ maior. Para aplicações com severas restrições de largura de banda (como ecossistemas IoT e distribuição de certificados digitais em larga escala), esse *overhead* constitui o principal desafio de adoção.
- **Dilithium3 vs. Dilithium2:** Apresenta um acréscimo de apenas $1,49\times$ em pk e $1,36\times$ em sig . Desse modo, a transição para o nível extra de segurança (de 128 para 192 bits quânticos) impõe um custo moderado de espaço adicional em disco ou rede.

5.5 LIMITAÇÕES DO EXPERIMENTO

As análises quantitativas deste estudo devem considerar as seguintes limitações experimentais e metodológicas:

Implementações assimétricas: O ECDSA utiliza o OpenSSL (C altamente otimizado com instruções SIMD), enquanto o Dilithium utiliza uma biblioteca em Python puro (*dilithium-py*). Portanto, a comparação de tempos absolutos não reflete o desempenho relativo real dos algoritmos em ambiente de produção; todas as razões de tempo devem ser interpretadas como artefatos das implementações de *software*, e não das propriedades intrínsecas dos esquemas criptográficos;

Signing determinístico: O uso de um par (sk, msg) fixo na Fase 2 do protocolo de medição produz invariavelmente o mesmo número de iterações no loop interno do algoritmo. Consequentemente, as estatísticas de *rejection sampling* obtidas refletem estritamente o comportamento de uma chave específica gerada no experimento, e não a distribuição estatística esperada sobre chaves aleatórias;

Mensagem reduzida e sem variação: O experimento utilizou mensagens estáticas de 214 bytes. Nessa escala, o tempo de execução do *signing* é amplamente dominado pelo processamento das estruturas de chave (como transformações NTT e multiplicações matriciais) e não pelo custo computacional do *hashing* da mensagem, tornando os tempos observados representativos para a maioria dos tamanhos práticos de payload;

Ambiente de execução não isolado: A medição via tempo de parede (*wall-clock time*) inclui variáveis espúrias como a latência de escalonamento do sistema operacional, o comportamento do *Garbage Collector* (GC) do Python e flutuações de memória *cache*. Em

cenários de produção, os tempos seriam mais estáveis mediante o uso de isolamento de processos e afinidade de CPU (*core pinning*);

Ausência de métricas de memória: O experimento limitou-se à análise de tempo e armazenamento em disco, não aferindo o consumo de memória RAM durante as operações de curva elíptica e redes polinomiais. Essa métrica é crítica para a viabilidade do Dilithium em ecossistemas de IoT e sistemas embarcados restritos, onde a implementação pura pode se mostrar inviável.

5.6 IMPLICAÇÕES PRÁTICAS PARA MIGRAÇÃO

A comparação entre ECDSA e Dilithium, à luz dos resultados, sugere as seguintes conclusões para planejamento de migração:

Tabela 9 – Comparativo prático entre ECDSA e Dilithium para decisão de adoção.
Fonte: Elaborado pelo autor.

Critério	ECDSA 256	P- ms	Dilithium2 ms	Dilithium3 ms	Vencedor Prático
Desempenho (C nativo)	~ 0,05 <i>sign</i>		~ 0,10 <i>sign</i>		ECDSA (mas <i>gap</i> < 5×)
Assinatura (bytes)	72		2.420		ECDSA
Chave pública (bytes)	64		1.312		ECDSA
Resistência quântica	Nenhuma		128 bits Q		Dilithium
Resistência a <i>nonce</i>	Não (risco)		Sim (determ.)		Dilithium
Padronização NIST	FIPS 186-5		FIPS 204 (2024)		FIPS 204 (2024) Ambos

Nota: Dados de desempenho baseados em estimativas de literatura para implementações em C nativo, contrastando com o ambiente controlado em Python do experimento principal.

Para sistemas com vida útil superior a 10 anos, o NIST recomenda iniciar imediatamente a migração para algoritmos pós-quânticos (National Institute of Standards and Technology, 2024). Para TLS, recomenda-se abordagem híbrida (Dilithium2 + ECDSA em paralelo) durante o período de transição, garantindo compatibilidade com clientes que não suportam algoritmos pós-quânticos. Dilithium2 é preferível ao Dilithium3 para uso geral, pois 128 bits de segurança quântica são considerados suficientes para a maioria das aplicações com horizonte até 2060.

6 CONCLUSÃO

Este trabalho apresentou um estudo comparativo teórico e experimental entre o esquema clássico ECDSA P-256 e o esquema pós-quântico CRYSTALS-Dilithium (ML-DSA, FIPS 204) nas variantes Dilithium2 e Dilithium3.

6.1 CONFIRMAÇÕES TEÓRICAS

Os tamanhos de chave e assinatura do Dilithium obtidos no experimento coincidem com exatidão com os valores do FIPS 204, validando a implementação utilizada. A estrutura do algoritmo *Sign*, particularmente o mecanismo de *rejection sampling* baseado no paradigma Fiat-Shamir com *Aborts*, foi confirmada experimentalmente, com a distinção entre os comportamentos de Dilithium2 (baixa taxa de rejeição) e Dilithium3 (alta taxa de rejeição para a chave de teste) explicando a diferença desproporcional nos tempos de *signing* ($2,99\times$) em relação a *keygen* e *verify* ($\sim 1,6\times$).

6.2 SOBRE O DESEMPENHO OBSERVADO

As razões de tempo entre Dilithium e ECDSA ($354\times$ para *Sign* do Dilithium2; $1.058\times$ para Dilithium3) refletem a assimetria de implementações: OpenSSL (C otimizado) vs. Python puro. Estimativas da literatura indicam razões de $\sim 2\text{--}4\times$ em implementações C nativas - *overhead* modesto para a maioria das aplicações interativas. O *overhead* de tamanho ($34\times$ para a assinatura do Dilithium2) é intrínseco ao esquema e independente da implementação, representando o principal desafio em aplicações com restrições de banda.

6.3 SOBRE O SIGNING DETERMINÍSTICO

Uma descoberta metodológica relevante foi que a assinatura do Dilithium é determinística para um par (sk, msg) fixo, de modo que as estatísticas de *rejection sampling* medidas refletem o comportamento de uma chave específica. Para medição estatisticamente válida de $E[K]$, seria necessário gerar chaves diferentes para cada iteração. Essa descoberta foi documentada como limitação e guia experimentos futuros.

6.4 PERSPECTIVA DE ADOÇÃO

O NIST recomenda migração imediata para sistemas com vida útil superior a 10 anos. Dilithium2 apresenta o melhor balanço entre segurança pós-quântica (128 bits quânticos), desempenho e tamanho, sendo a variante recomendada para uso geral. A abordagem híbrida (Dilithium + ECDSA) é indicada para o período de transição, garantindo interoperabilidade retrocompatível.

6.5 TRABALHOS FUTUROS

Como extensões naturais deste trabalho, propõe-se:

- (a) repetição do experimento com implementações C nativas (`liboqs`) para comparação justa de desempenho;
- (b) medição de $E[K]$ com chaves variadas por iteração;
- (c) inclusão de Dilithium5 e SPHINCS+ para cobertura completa dos padrões FIPS 204–205;
- (d) análise de consumo de memória RAM para avaliação de viabilidade em dispositivos embarcados;
- (e) *benchmark* de *throughput* em servidor com múltiplos processos paralelos.

REFERÊNCIAS

- ALBRECHT, M. et al. Estimate all the lwe, ntru schemes! In: **Security and Cryptography for Networks – SCN 2018**. [S.l.]: Springer, 2018. (Lecture Notes in Computer Science, v. 11035), p. 351–367.
- Anthropic. **Claude: Large Language Model and AI Assistant**. 2026. Modelo de inteligência artificial, utilizado como ferramenta de suporte para organização. Disponível em: <https://claude.ai/>.
- DUCAS, L. et al. Crystals-dilithium: A lattice-based digital signature scheme. **IACR Transactions on Cryptographic Hardware and Embedded Systems**, v. 2018, n. 1, p. 238–268, 2018.
- Google. **Gemini: Large Language Model and AI Collaborator**. 2026. Modelo de inteligência artificial, utilizado como ferramenta de suporte para formatação do documento LaTeX. Disponível em: <https://gemini.google.com/>.
- LYUBASHEVSKY, V. Fiat-shamir with aborts: Applications to lattice and factoring-based signatures. In: **Advances in Cryptology – ASIACRYPT 2009**. [S.l.]: Springer, 2009. (Lecture Notes in Computer Science, v. 5912), p. 598–616.
- MOSCA, M. Cybersecurity in an era with quantum computers: will we be ready? **IEEE Security & Privacy**, v. 16, n. 5, p. 38–41, 2018.
- National Institute of Standards and Technology. **Digital Signature Standard (DSS)**. Gaithersburg, MD, 2023. Disponível em: <https://csrc.nist.gov/pubs/fips/186-5/final>.
- National Institute of Standards and Technology. **Module-Lattice-Based Digital Signature Standard (ML-DSA)**. Gaithersburg, MD, 2024. Disponível em: <https://csrc.nist.gov/pubs/fips/204/final>.
- POPE, G. **dilithium-py: Pure Python implementation of CRYSTALS-Dilithium**. 2023. GitHub. Disponível em: <https://github.com/GiacomoPope/dilithium-py>.
- REGEV, O. On lattices, learning with errors, random linear codes, and cryptography. **Journal of the ACM**, v. 56, n. 6, 2009.
- ROETTELIER, M. et al. Quantum resource estimates for computing elliptic curve discrete logarithms. In: **Advances in Cryptology – ASIACRYPT 2017**. [S.l.]: Springer, 2017. (Lecture Notes in Computer Science, v. 10625), p. 241–270.
- SHOR, P. W. Algorithms for quantum computation: discrete logarithms and factoring. In: **Proceedings 35th Annual Symposium on Foundations of Computer Science**. [S.l.]: IEEE, 1994. p. 124–134.